

Министерство науки и высшего образования  
Российской Федерации  
ФГАОУ ВО «УрФУ имени первого Президента России Б.Н. Ельцина»  
ИРИТ-РТФ  
Школа магистратуры

Лабораторная работа №8

«Введение в работу с XML, JSON, HTML и Excel»

Преподаватель

Архипов Н.А.

Студент

Жунёв А.А.

Группа РИМ-150950

Екатеринбург 2026

## Цель работы

Получить практические навыки работы с распространенными форматами данных в Java: XML, JSON, HTML и Excel. Изучить способы создания, чтения, поиска, изменения и удаления данных в файлах, а также обработку ошибок при работе с файловыми и сетевыми источниками.

Ссылка на репозиторий: [https://github.com/2made1ra/JAVA\\_SBER\\_2026](https://github.com/2made1ra/JAVA_SBER_2026)

Код лабораторной работы расположен в пакете:

`java-core-2026/src/lab8`

## 1. Описание задания

В рамках лабораторной работы необходимо:

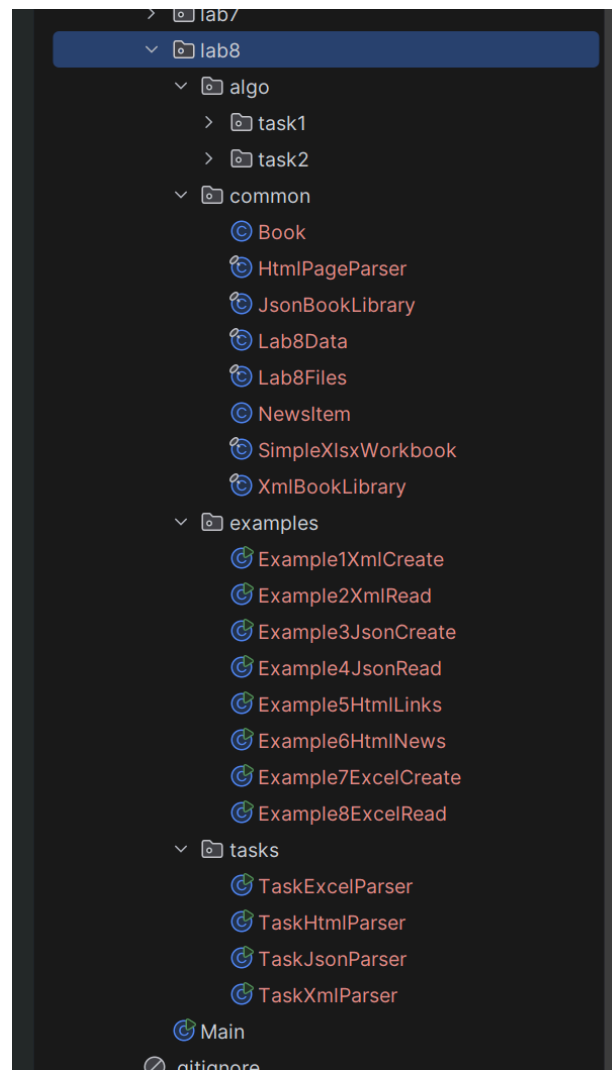
1. Выполнить примеры из раздела 1 методических материалов:
  - создание и чтение XML-файла;
  - создание и чтение JSON-файла;
  - парсинг HTML-страниц;
  - создание и чтение Excel-файла.
2. Доработать XML-парсер:
  - подготовить собственный файл в формате XML;
  - добавить запись новых книг;
  - добавить поиск книг по автору и году издания;
  - реализовать удаление книги по названию.
3. Доработать JSON-парсер:
  - подготовить собственный файл в формате JSON;
  - добавить поиск книг по автору;
  - добавить добавление новой книги;
  - реализовать удаление книги по названию.
4. Доработать HTML-парсер:
  - сохранять полученные новости в файл;
  - добавить обработку ошибок подключения и повторные попытки загрузки страницы.
5. Доработать Excel-парсер:
  - улучшить сообщения об ошибках при неправильном формате файла или отсутствии нужного листа.
6. Решить две алгоритмические задачи. Вместо задач Timus были реализованы две задачи CodeRun:
  - «НВП с восстановлением ответа»;
  - «Расстояние по Левенштейну».

## 2. Ход выполнения

### 2.1. Общая обвязка лабораторной работы

Для повторяющихся операций были добавлены вспомогательные классы:

- `java-core-2026/src/lab8/common/Book.java` — модель книги с полями `title`, `author`, `year`.
- `java-core-2026/src/lab8/common/NewsItem.java` — модель новости с датой и заголовком.
- `java-core-2026/src/lab8/common/Lab8Data.java` — демонстрационные данные для книг и товаров.
- `java-core-2026/src/lab8/common/Lab8Files.java` — создание временных каталогов и родительских директорий для файлов.

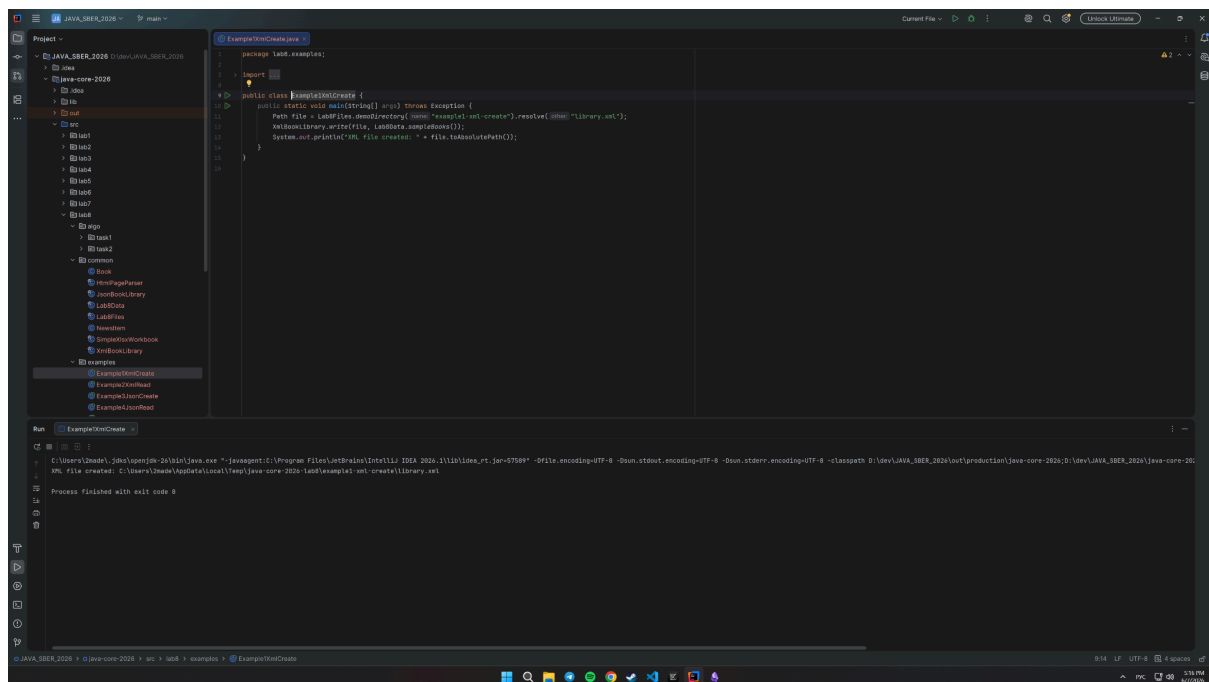


## 2.2. Пример 1. Создание XML-файла

Файл с решением:

java-core-2026/src/lab8/examples/Example1XmlCreate.java

В примере создается файл **library.xml**. Для записи используется класс **XmlBookLibrary**, который формирует DOM-документ с корневым элементом **library** и элементами **book**.

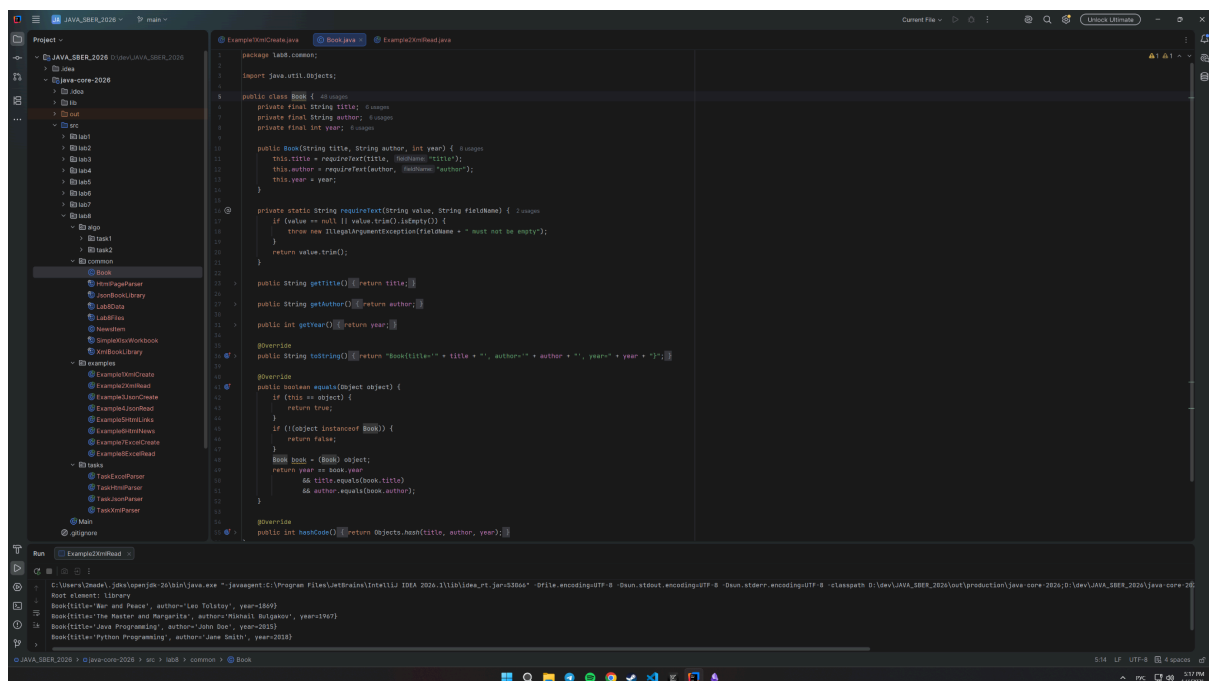
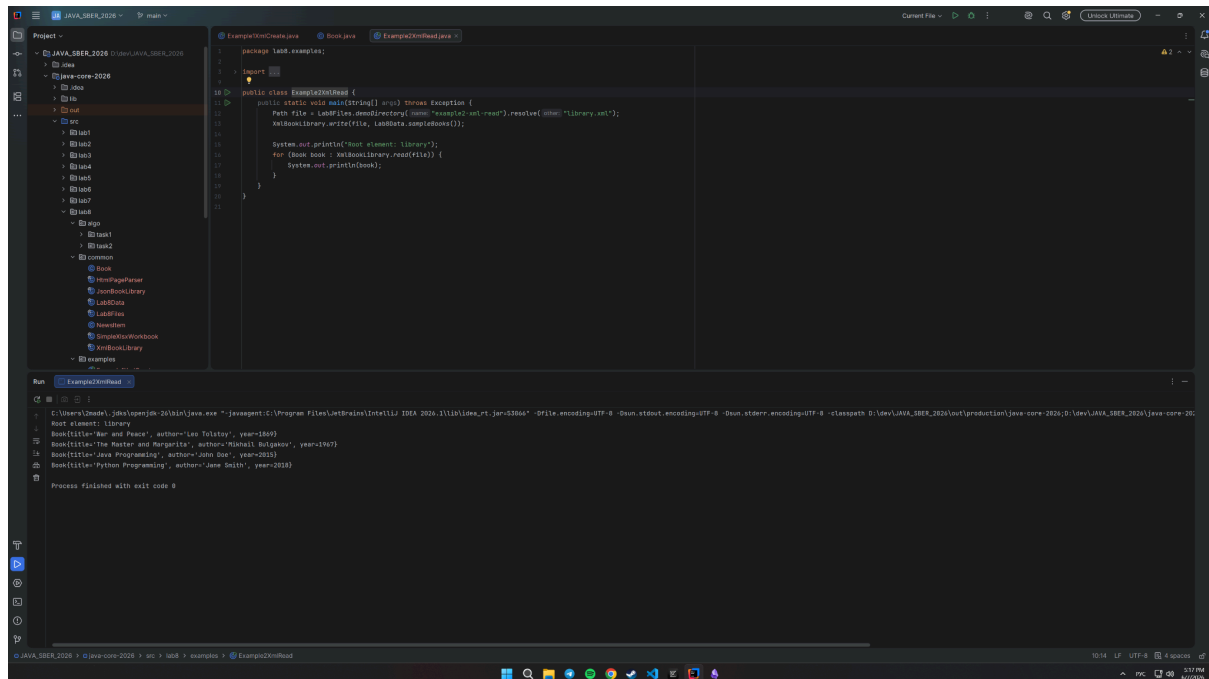


### 2.3. Пример 2. Чтение XML-файла

Файл с решением:

java-core-2026/src/lab8/examples/Example2XmlRead.java

Программа создает демонстрационный XML-файл, затем читает его через DOM-парсер и выводит список книг.

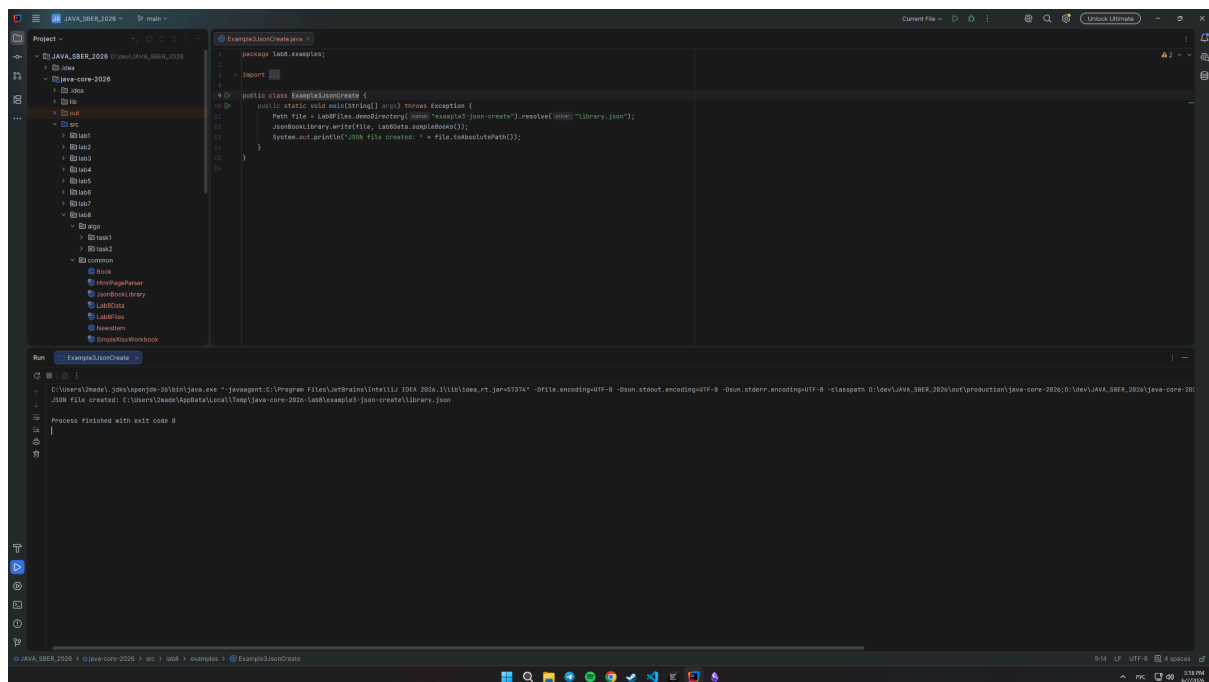


## 2.4. Пример 3. Создание JSON-файла

Файл с решением:

java-core-2026/src/lab8/examples/Example3JsonCreate.java

Программа записывает массив книг в JSON-файл. Так как проект выполняется без Maven-зависимостей, JSON создается собственной небольшой обвязкой [JsonBookLibrary](#).

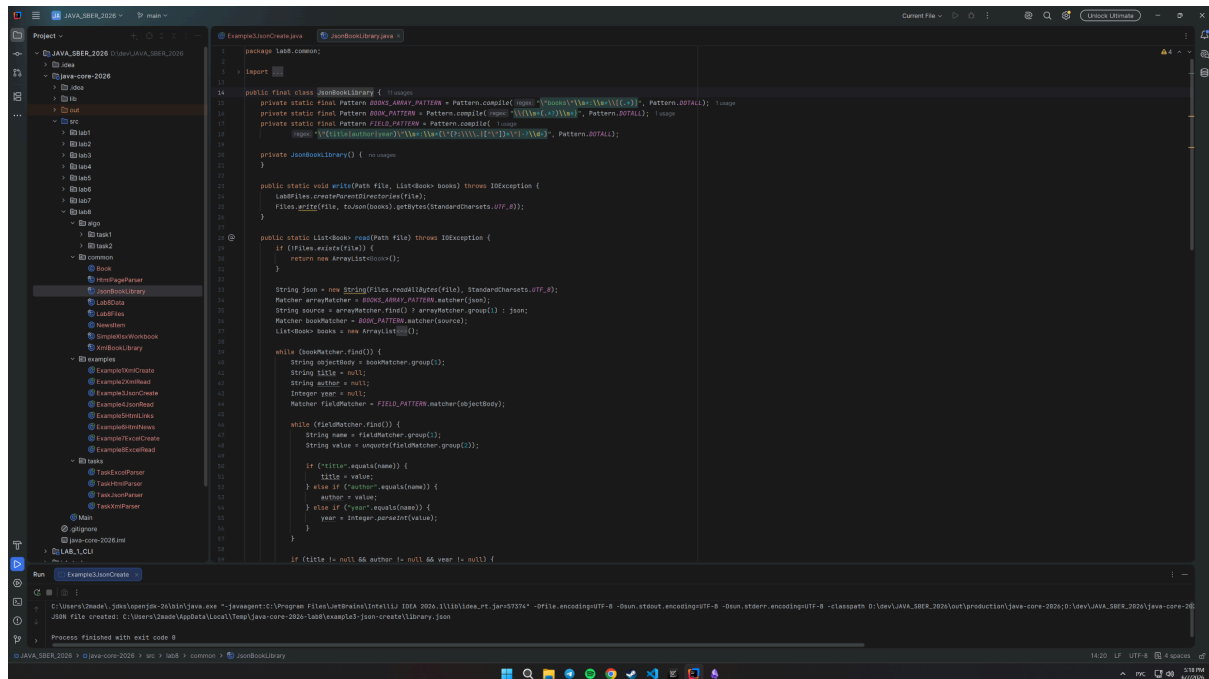


## 2.5. Пример 4. Чтение JSON-файла

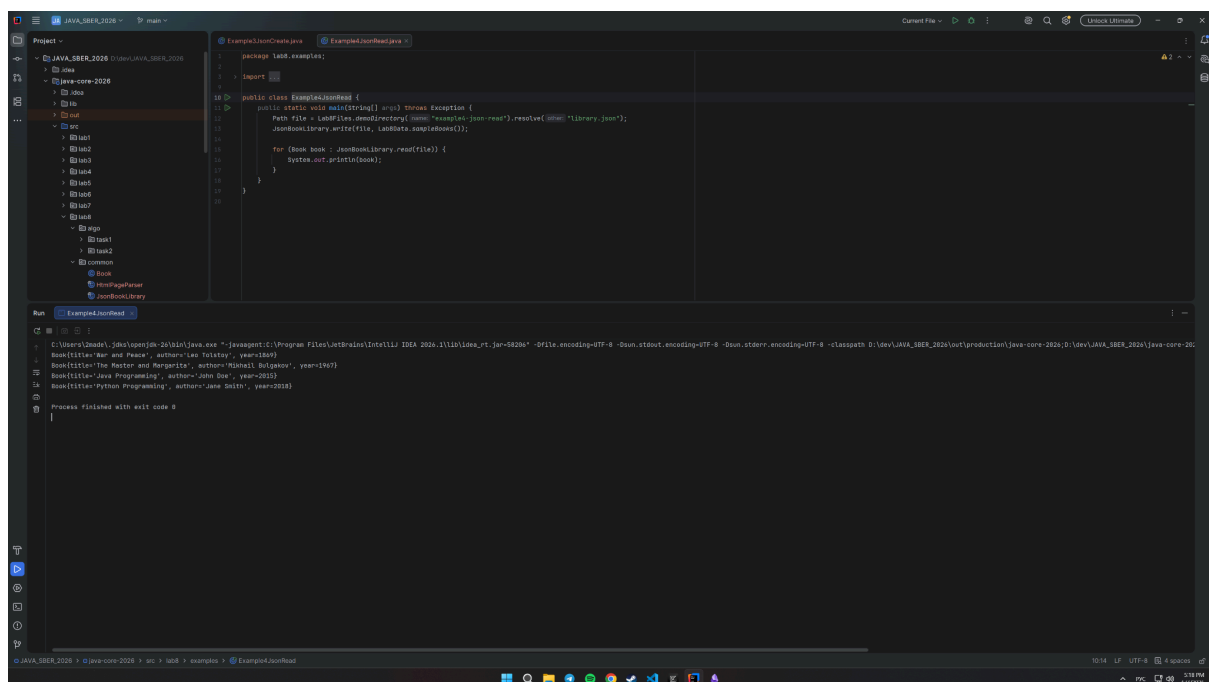
Файл с решением:

java-core-2026/src/lab8/examples/Example4JsonRead.java

Программа читает JSON-файл и преобразует объекты из массива **books** в экземпляры класса **Book**.



```
1 package lab8.common;
2
3 import java.io.*;
4 import java.util.*;
5
6 public final class JsonBookLibrary {
7     private static final Pattern BOOK_ARRAY_PATTERN = Pattern.compile("(\\[[{\\s]*\\s*\\{.*?\\}\\s*,*\\s*\\{.*?\\}\\s*,*\\s*\\}\\s*", Pattern.DOTALL);
8     private static final Pattern BOOK_PATTERN = Pattern.compile("(\\{.*?\\})", Pattern.DOTALL);
9     private static final Pattern FIELD_PATTERN = Pattern.compile("(\\s*\\{.*?\\}\\s*", Pattern.DOTALL);
10
11     private JsonBookLibrary() {}
12
13     public static void write(Path file, List<Book> books) throws IOException {
14         LoadFiles.createParentDirectories(file);
15         Files.write(file, books.toJson().getBytes(StandardCharsets.UTF_8));
16     }
17
18     public static List<Book> read(Path file) throws IOException {
19         if (!Files.exists(file)) {
20             return new ArrayList<>();
21         }
22
23         String json = new String(Files.readAllBytes(file), StandardCharsets.UTF_8);
24         Matcher arrayMatcher = BOOK_ARRAY_PATTERN.matcher(json);
25         String source = arrayMatcher.find() ? arrayMatcher.group(1) : json;
26         Matcher bookMatcher = BOOK_PATTERN.matcher(source);
27         List<Book> books = new ArrayList<>();
28
29         while (bookMatcher.find()) {
30             String objectJson = bookMatcher.group(1);
31             String title = null;
32             String author = null;
33             Integer year = null;
34             Matcher fieldMatcher = FIELD_PATTERN.matcher(objectJson);
35
36             while (fieldMatcher.find()) {
37                 String name = fieldMatcher.group(1);
38                 String value = quote(fieldMatcher.group(2));
39
40                 if ("title".equals(name)) {
41                     title = value;
42                 } else if ("author".equals(name)) {
43                     author = value;
44                 } else if ("year".equals(name)) {
45                     year = Integer.parseInt(value);
46                 }
47             }
48             if (title != null && author != null && year != null) {
49                 books.add(new Book(title, author, year));
50             }
51         }
52     }
53 }
```



```
1 package lab8.examples;
2
3 import java.io.*;
4 import java.util.*;
5
6 public final class Example4JsonRead {
7     private static void main(String[] args) throws Exception {
8         Path file = LoadFiles.demoDirectory("examples/json-read").resolve("lib8/library.json");
9         JsonBookLibrary.write(file, LoadData.sampleBooks());
10
11         for (Book book : JsonBookLibrary.read(file)) {
12             System.out.println(book);
13         }
14     }
15 }
```

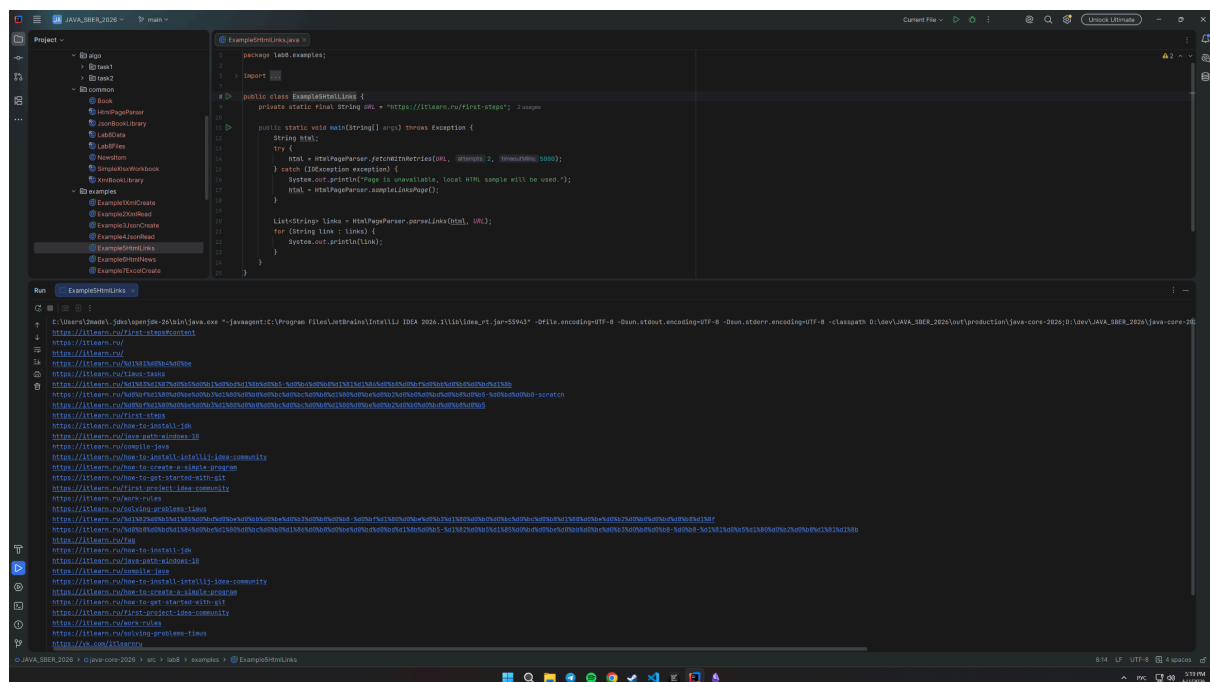


## 2.6. Пример 5. Парсинг ссылок HTML-страницы

Файл с решением:

java-core-2026/src/lab8/examples/Example5HtmlLinks.java

Пример пытается загрузить страницу <https://itlearn.ru/first-steps>, извлекает ссылки из HTML-кода и приводит относительные ссылки к абсолютному виду. При недоступности страницы используется локальная демонстрационная HTML-строка.

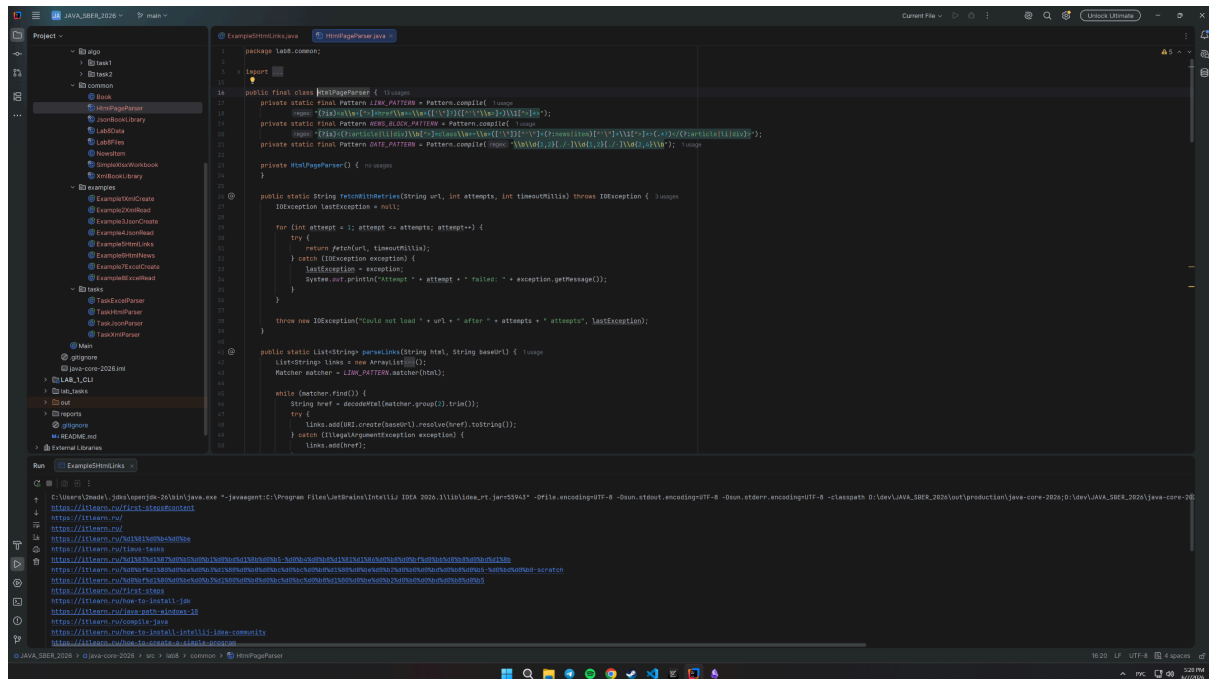


## 2.7. Пример 6. Парсинг новостей HTML-страницы

Файл с решением:

java-core-2026/src/lab8/examples/Example6HtmlNews.java

Программа извлекает новости из HTML-кода: дату и заголовок. При ошибке подключения используется локальный HTML-пример.



```
package lab.common;

import java.io.IOException;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.List;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public final class HtmlPageParser {
    private static final Pattern LINK_PATTERN = Pattern.compile("<a href='(.*?)'>(.*?)</a>");
    private static final Pattern DATE_PATTERN = Pattern.compile("<div class='news-date'>(.*?)</div>");
    private static final Pattern TITLE_PATTERN = Pattern.compile("<div class='news-title'>(.*?)</div>");

    private HtmlPageParser() {}

    public static String fetchHtml(String url, int attempts, int timeoutMillis) throws IOException {
        IOException lastException = null;

        for (int attempt = 1; attempt <= attempts; attempt++) {
            try {
                return fetch(url, timeoutMillis);
            } catch (IOException exception) {
                lastException = exception;
                System.out.println("attempt " + attempt + " failed: " + exception.getMessage());
            }
        }

        throw new IOException("could not load " + url + " after " + attempts + " attempts", lastException);
    }

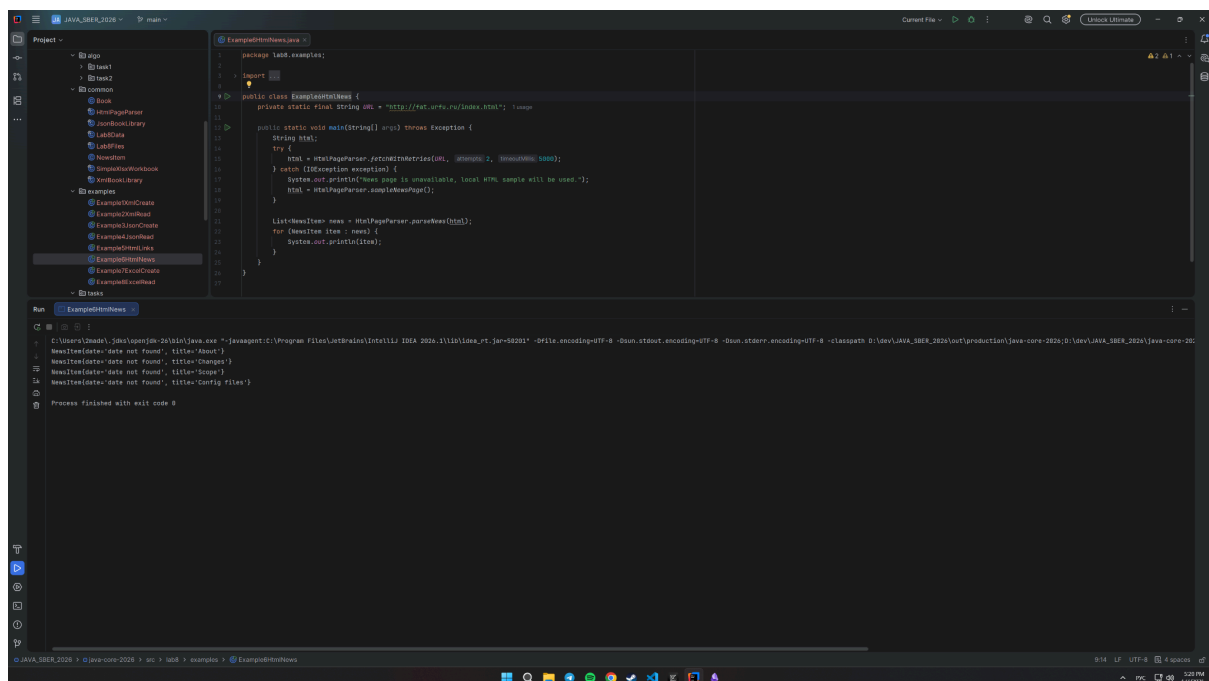
    public static List<String> parseNews(String html, String baseUrl) {
        List<String> lines = new ArrayList<>();
        Matcher matcher = LINK_PATTERN.matcher(html);

        while (matcher.find()) {
            String href = decodeHtml(matcher.group(1).trim());
            try {
                lines.add(baseUrl + href);
            } catch (IllegalArgumentException exception) {
                lines.add(href);
            }
        }

        return lines;
    }

    private static String fetch(String url, int timeoutMillis) throws IOException {
        URL urlObj = new URL(url);
        HttpURLConnection conn = (HttpURLConnection) urlObj.openConnection();
        conn.setConnectTimeout(timeoutMillis);
        conn.setReadTimeout(timeoutMillis);
        conn.setRequestMethod("GET");
        int responseCode = conn.getResponseCode();
        if (responseCode != HttpURLConnection.HTTP_OK) {
            throw new IOException("HTTP error: " + responseCode);
        }
        return conn.getInputStream().readAllBytes();
    }

    private static String decodeHtml(String href) {
        return href.replace("&#x2D;", "-").replace("&#x26;", "&").replace("&#x27;", "'").replace("&#x22;", "\"");
    }
}
```



```
package lab.examples;

import lab.common.HtmlPageParser;

public class Example6HtmlNews {
    private static final String URL = "http://net.urfu.ru/index.html";

    public static void main(String[] args) throws Exception {
        String html;
        try {
            html = HtmlPageParser.fetchHtml(URL, 10, 10000);
        } catch (IOException exception) {
            System.out.println("main page is unavailable, local HTML sample will be used.");
            html = HtmlPageParser.sampleHtmlPage();
        }

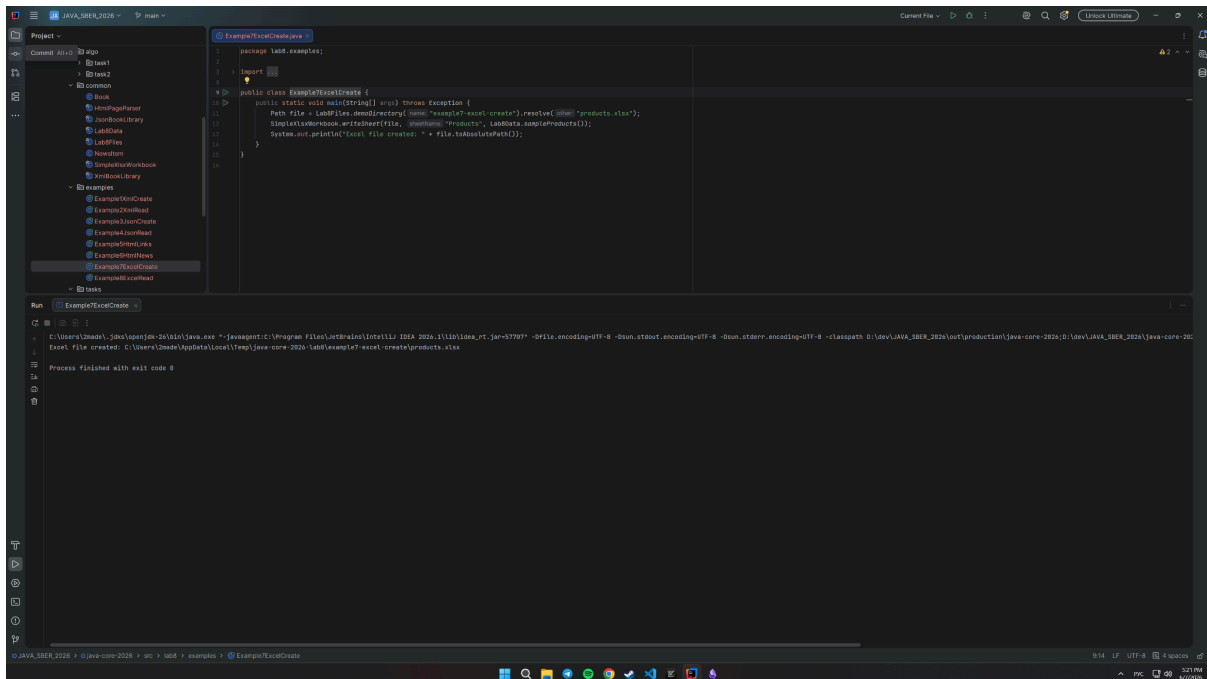
        List<String> news = HtmlPageParser.parseNews(html);
        for (String item : news) {
            System.out.println(item);
        }
    }
}
```

## 2.8. Пример 7. Создание Excel-файла

Файл с решением:

java-core-2026/src/lab8/examples/Example7ExcelCreate.java

Программа создает файл **products.xlsx** и записывает в него таблицу товаров. Для работы без внешних зависимостей реализован класс **SimpleXlsxWorkbook**, который формирует XLSX как ZIP-архив с необходимыми XML-частями.

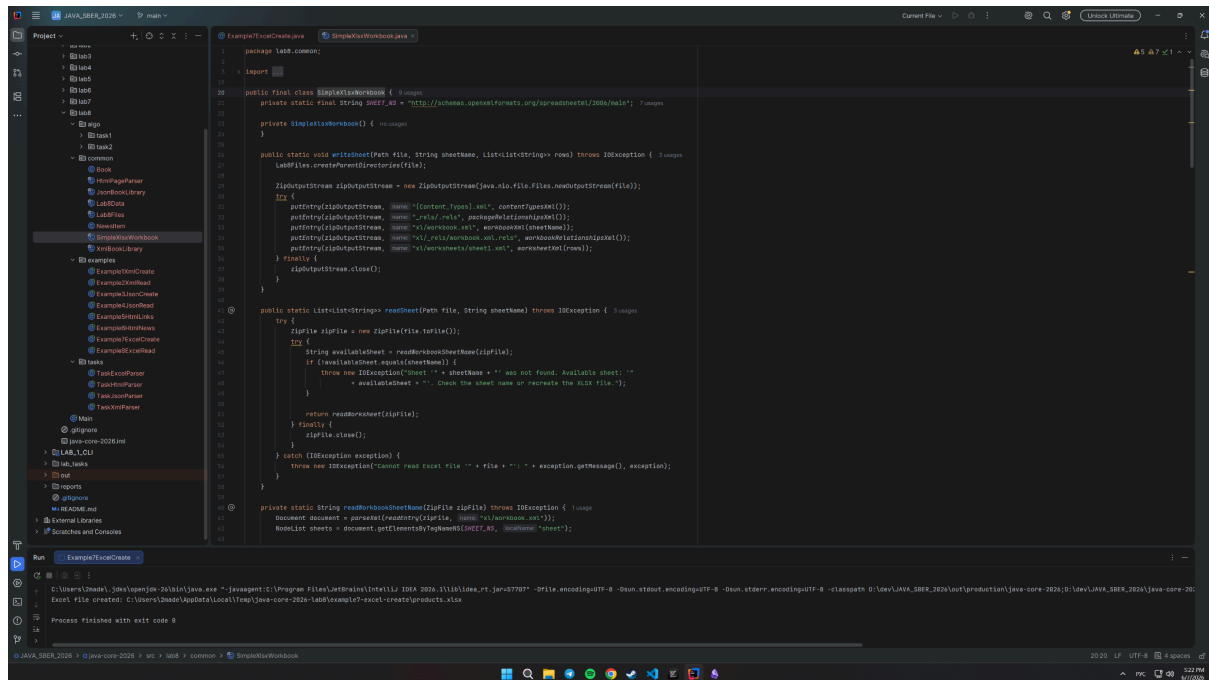


## 2.9. Пример 8. Чтение Excel-файла

Файл с решением:

java-core-2026/src/lab8/examples/Example8ExcelRead.java

Программа читает созданный XLSX-файл и выводит строки таблицы.



```
package lab8.common;

import java.io.*;
import java.util.*;

public final class SimpleXlsxWorkbook {
    private static final String SHEET_NAME = "http://schemas.openxmlformats.org/spreadsheetml/2006/main";

    private SimpleXlsxWorkbook() {}

    public static void writeSheet(Path file, String sheetName, List<List<String>> rows) throws IOException {
        Lab8Files.createParentDirectories(file);

        ZipOutputStream zipOutputStream = new ZipOutputStream(new FileOutputStream(file));

        try {
            zipOutputStream.putNextEntry(new ZipEntry(SHEET_NAME));
            zipOutputStream.putNextEntry(new ZipEntry(SHEET_NAME));
            zipOutputStream.putNextEntry(new ZipEntry(SHEET_NAME));
            zipOutputStream.putNextEntry(new ZipEntry(SHEET_NAME));
            zipOutputStream.putNextEntry(new ZipEntry(SHEET_NAME));
        } finally {
            zipOutputStream.close();
        }

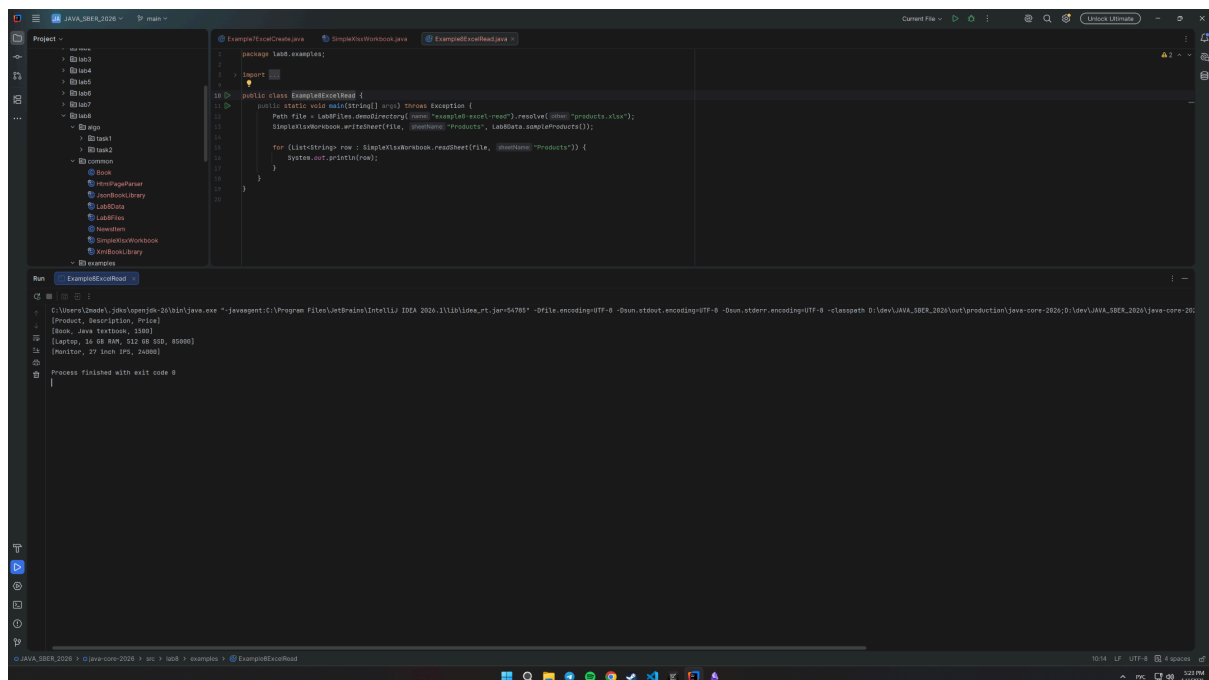
        public static List<List<String>> readSheet(Path file, String sheetName) throws IOException {
            try {
                ZipFile zipFile = new ZipFile(file.toFile());

                String availableSheet = readWorkbookSheetNames(zipFile);

                if (!availableSheet.equals(sheetName)) {
                    throw new IOException("Sheet '" + sheetName + "' was not found. Available sheet: '" + availableSheet + "' Check the sheet name or recreate the xlsx file.");
                }

                return readWorkbookSheet(zipFile);
            } finally {
                zipFile.close();
            }
        } catch (IOException exception) {
            throw new IOException("Cannot read excel file '" + file + "' : " + exception.getMessage(), exception);
        }

        private static String readWorkbookSheetNames(ZipFile zipFile) throws IOException {
            Document document = parse(zipFile, SHEET_NAME);
            NodeList sheets = document.getElementsByTagName(SHEET_NAME);
        }
    }
}
```



```
package lab8.examples;

import java.io.*;
import java.util.*;

public class SimpleExcelRead {
    public static void main(String[] args) throws IOException {
        Path file = Lab8Files.workspace().resolve("example8-excel-read").resolve("products.xlsx");
        SimpleXlsxWorkbook workbook = new SimpleXlsxWorkbook();
        List<List<String>> rows = workbook.readSheet(file, "Products");
        for (List<String> row : rows) {
            System.out.println(row);
        }
    }
}
```

## 3. Задания для самостоятельного выполнения

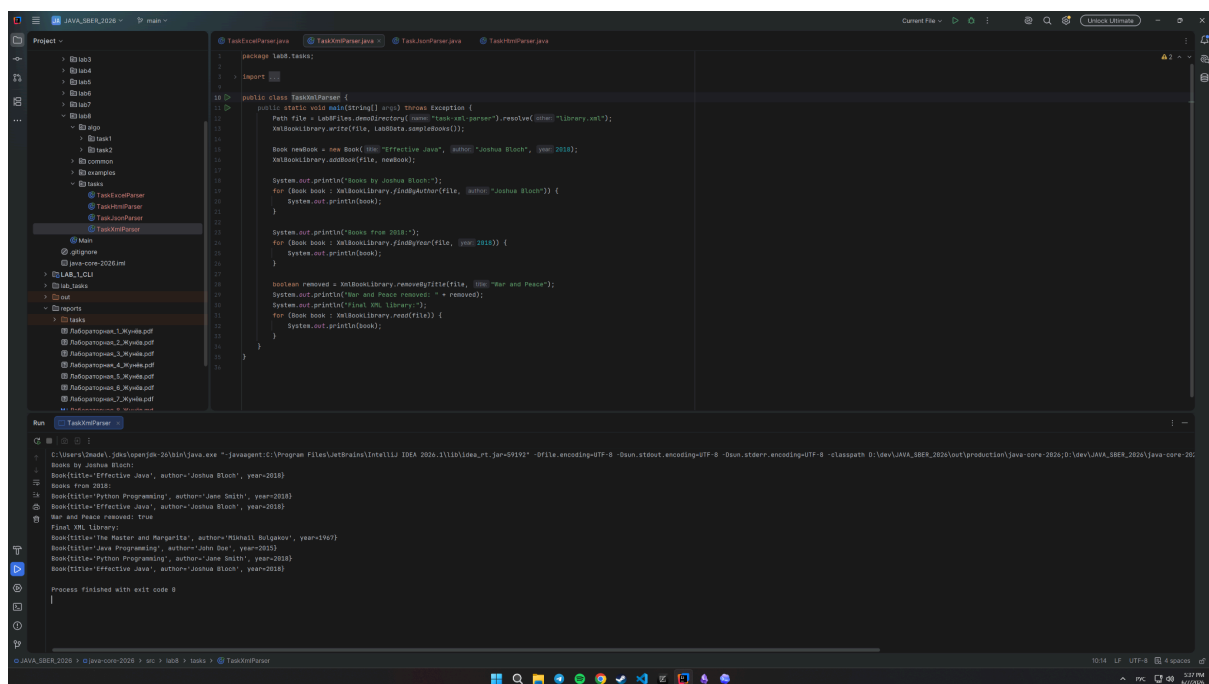
### 3.1. Доработка XML-парсера

Файл с решением:

java-core-2026/src/lab8/tasks/TaskXmlParser.java

Реализованы операции:

- создание XML-файла со списком книг;
- добавление новой книги;
- поиск книг по автору;
- поиск книг по году издания;
- удаление книги по названию.



The screenshot shows an IDE window with the following components:

- Project Explorer:** Shows a project structure with folders 'lab3' through 'lab8', 'src', 'common', 'tasks', and 'reports'. The 'tasks' folder is expanded, showing 'TaskXmlParser.java'.
- Code Editor:** Displays the content of 'TaskXmlParser.java'. The code includes a package declaration, imports, and a public class 'TaskXmlParser' with a main method. The main method performs several operations: it creates a new book, adds it to a library, prints books by author, prints books from a specific year, removes a book by title, and prints the remaining books.
- Run Console:** Shows the output of the program. It displays the list of books in the library, the books found by author and year, and the result of removing a book. The output ends with 'Process finished with exit code 0'.

## 3.2. Доработка JSON-парсера

Файл с решением:

java-core-2026/src/lab8/tasks/TaskJsonParser.java

Реализованы операции:

- создание JSON-файла со списком книг;
- добавление книги в массив;
- поиск книг по автору;
- удаление книги по названию;
- повторное чтение итогового списка.

The screenshot shows an IDE with the following components:

- Project Explorer:** Shows a project structure with folders like 'src', 'lab1' through 'lab8', 'examples', and 'tasks'. The 'tasks' folder is expanded, showing 'TaskJsonParser'.
- Editor:** Displays the code for 'TaskJsonParser.java'. The code includes:
  - Package declaration: `package lab8.tasks;`
  - Imports: `import java.io.*;`
  - Class definition: `public class TaskJsonParser {`
  - Method `main`:
    - Creates a `Path` for the JSON file: `Path file = Paths.get(directoryName, "task-json-parser").resolve(Paths.get("library.json"));`
    - Writes the initial JSON data to the file: `JsonBookLibrary.write(file, loadData.sampleBooks());`
    - Creates a new book: `Book newBook = new Book(100, "Clean Code", author="Robert Martin", year=2008);`
    - Adds the new book: `JsonBookLibrary.addBook(file, newBook);`
    - Prints books by author: `System.out.println("Books by Robert Martin:");` followed by a loop: `for (Book book : jsonBookLibrary.findAuthor(file, author="Robert Martin")) { System.out.println(book); }`
    - Removes a book: `boolean removed = jsonBookLibrary.removeByTitle(file, title="Python Programming");`
    - Prints the updated JSON library: `System.out.println("Python Programming removed: " + removed);`
    - Prints the JSON library: `System.out.println("Final JSON Library:");` followed by a loop: `for (Book book : jsonBookLibrary.read(file)) { System.out.println(book); }`
  - Method `main` ends with `}`
- Run Console:** Shows the output of the program:

```
Books by Robert Martin:
Book{title=Clean Code, author=Robert Martin, year=2008}
Python Programming removed: true
Final JSON Library:
Book{title=Ham and Cheese, author=Leo Tolstoy, year=1869}
Book{title=The Master and Margarita, author=Michael Bulgakov, year=1967}
Book{title=Java Programming, author=John Doe, year=2015}
Book{title=Clean Code, author=Robert Martin, year=2008}
```

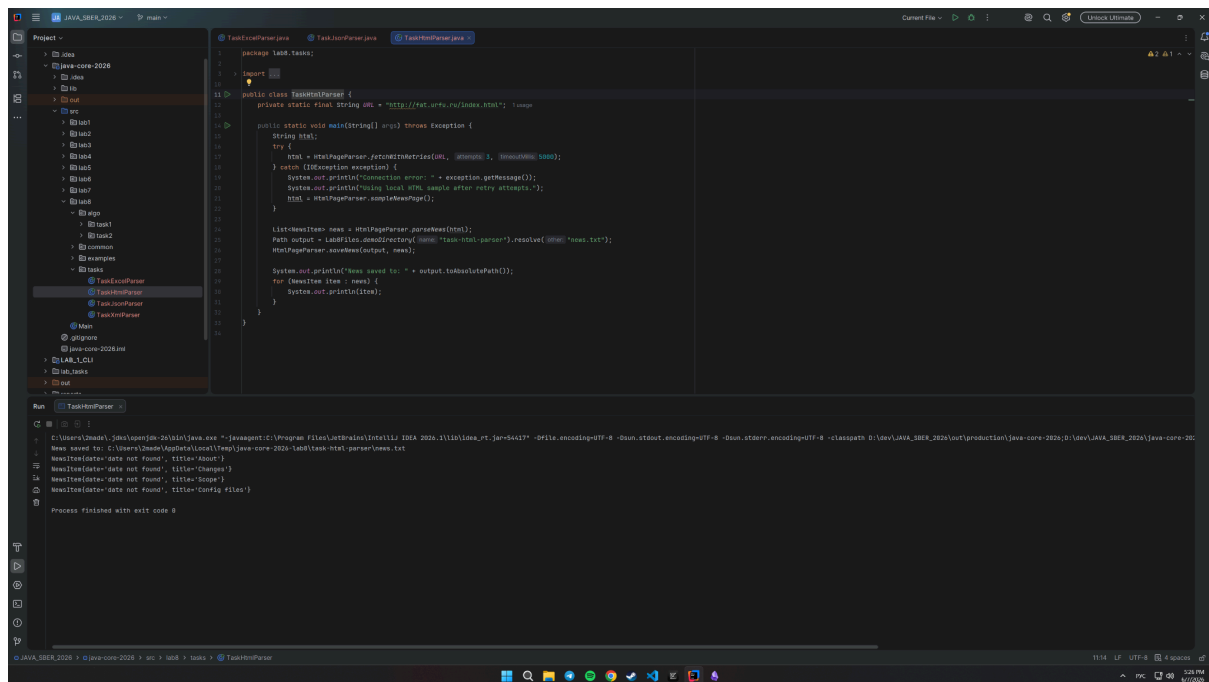
### 3.3. Доработка HTML-парсера

Файл с решением:

java-core-2026/src/lab8/tasks/TaskHtmlParser.java

Реализованы:

- три попытки загрузки страницы <http://fat.urfu.ru/index.html>;
- обработка ошибки подключения;
- переход к локальному HTML-примеру при недоступности сайта;
- сохранение списка новостей в файл [news.txt](#).



The screenshot shows an IDE with the following components:

- Project Explorer:** Shows the project structure with folders like 'src', 'lab1', 'lab2', 'lab3', 'lab4', 'lab5', 'lab6', 'lab7', 'lab8', 'app', 'task1', 'task2', 'common', 'examples', and 'tasks'. The 'TaskHtmlParser' class is selected under 'tasks'.
- Code Editor:** Displays the source code of 'TaskHtmlParser.java'. The code includes a package declaration, imports, a static URL, a main method with a try-catch block for connection attempts, and logic to save news items to a file named 'news.txt'.
- Run Console:** Shows the output of the program execution. It indicates that the news was saved to a file and lists several news items with their dates and titles, such as 'News saved to: C:\Users\Ivanov\AppData\Local\Temp\java-core-2026-lab8-task-html-parser\news.txt' and 'NewsItem(date= date not found, title= About)'.

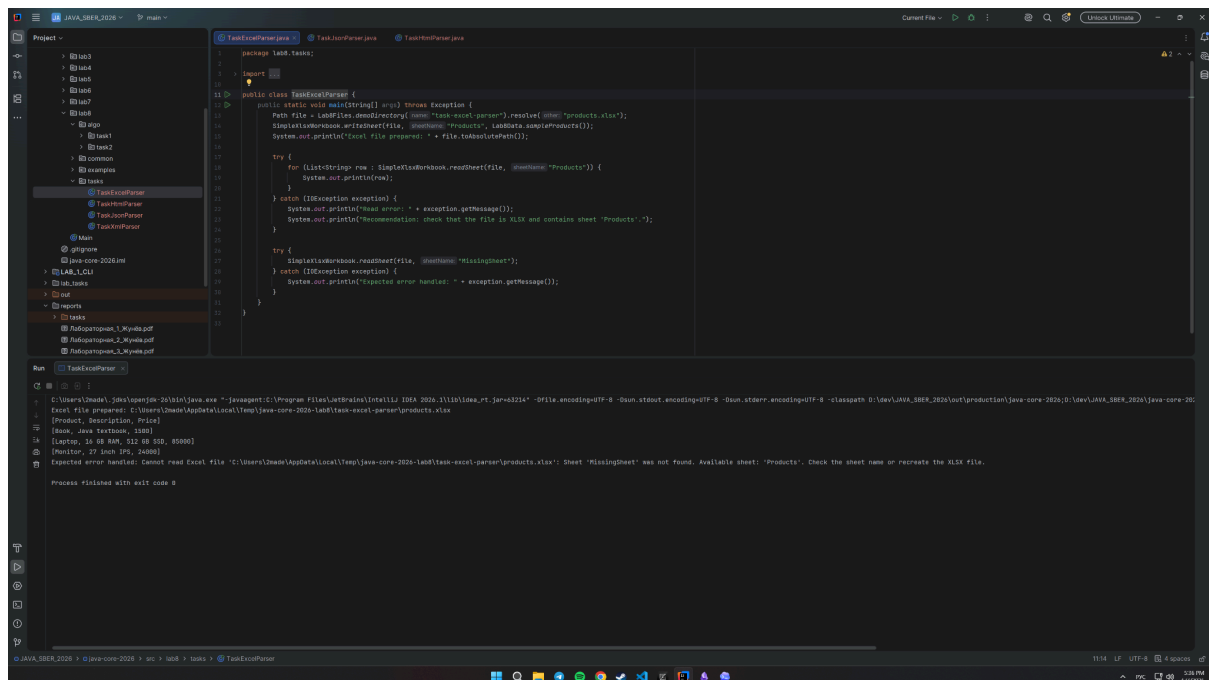
## 3.4. Доработка Excel-парсера

Файл с решением:

java-core-2026/src/lab8/tasks/TaskExcelParser.java

Реализованы:

- создание XLSX-файла с листом **Products**;
- чтение строк таблицы;
- обработка ошибки при попытке прочитать отсутствующий лист;
- вывод рекомендации по исправлению ошибки.



The screenshot shows an IDE with the following components:

- Project Explorer:** Shows a project structure with folders 'lab3' through 'lab8'. Under 'lab8', there are files 'TaskExcelParser.java', 'TaskExcelParserTest.java', and 'TaskExcelParser.java'.
- Code Editor:** Displays the content of 'TaskExcelParser.java'. The code is as follows:

```
1 package lab8.tasks;
2
3 import java.io.*;
4 import java.nio.file.*;
5 import org.apache.poi.xssf.usermodel.XSSFWorkbook;
6
7 public class TaskExcelParser {
8     public static void main(String[] args) throws Exception {
9         Path file = LabFiles.getDefault().resolve("products.xlsx");
10         SimpleWorkbook.writeSheet(file, "Products", LabData.sampleProducts());
11         System.out.println("Excel file prepared: " + file.toAbsolutePath());
12
13         try {
14             for (List<String> row : SimpleWorkbook.readSheet(file, "Products")) {
15                 System.out.println(row);
16             }
17         } catch (IOException exception) {
18             System.out.println("read error: " + exception.getMessage());
19             System.out.println("Recommendation: check that the file is XLSX and contains sheet 'Products'");
20         }
21
22         try {
23             SimpleWorkbook.readSheet(file, "MissingSheet");
24         } catch (IOException exception) {
25             System.out.println("read error handled: " + exception.getMessage());
26         }
27     }
28 }
```
- Run Console:** Shows the output of the program execution:

```
1 C:\Users\Ishana\Development> java.exe -javaagent:C:\Program Files\Java\jdk-20.0.2\jre\lib\rt.jar=42214 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encoding=UTF-8 -classpath 0:\dev\JAVA_SBER_2026\out\production\java-core-2026;0:\dev\JAVA_SBER_2026\java-core-2026\
Excel file prepared: C:\Users\Ishana\AppData\Local\Temp\java-core-2026-lab8-task-excel-parser\products.xlsx
[Product, Description, Price]
[book, Java textbook, 1500]
[Laptop, 16 inch, 8GB RAM, 512 GB SSD, $5000]
[Monitor, 27 inch IPS, 24000]
Expected error handled: Cannot read Excel file 'C:\Users\Ishana\AppData\Local\Temp\java-core-2026-lab8-task-excel-parser\products.xlsx': Sheet 'MissingSheet' was not found. Available sheet: 'Products'. Check the sheet name or recreate the XLSX file.
Process finished with exit code 0
```



## 4 Алгоритмические задания

### 4.1. CodeRun. «НВП с восстановлением ответа»

Ссылка на задачу:

<https://coderun.yandex.ru/problem/nvp-with-response-recovery/description>

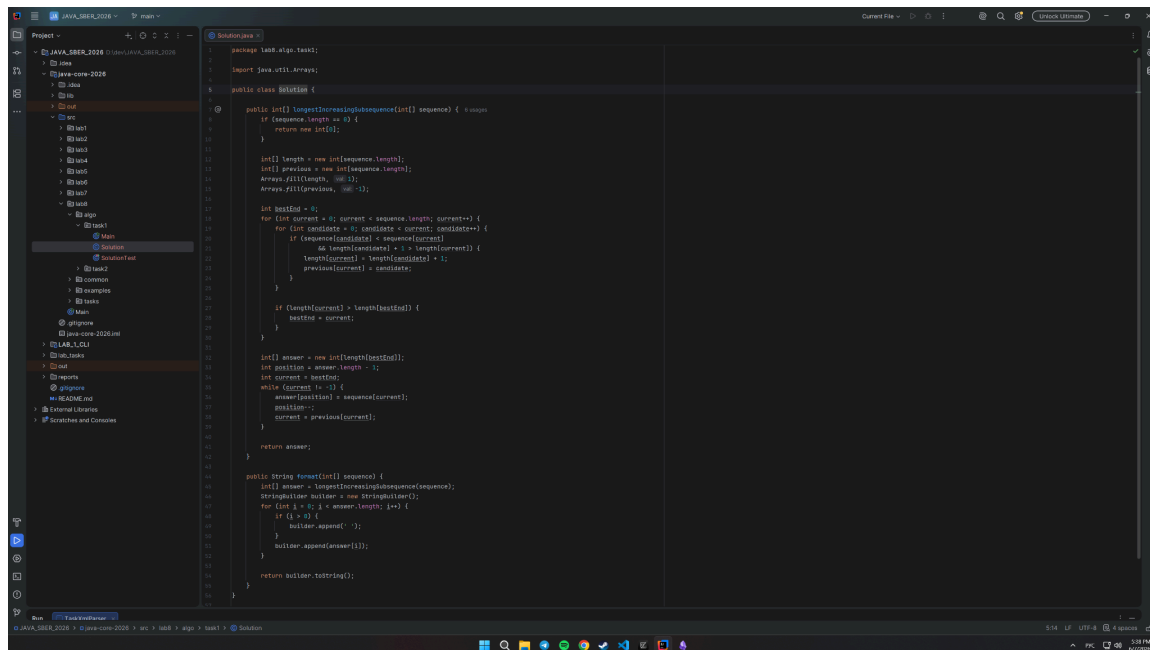
Файлы с решением:

java-core-2026/src/lab8/algo/task1/Main.java

java-core-2026/src/lab8/algo/task1/Solution.java

java-core-2026/src/lab8/algo/task1/SolutionTest.java

По условию требуется вывести любую наибольшую возрастающую подпоследовательность. В решении используется динамическое программирование за  $O(n^2)$ : для каждого элемента хранится длина лучшей возрастающей подпоследовательности, которая заканчивается в этой позиции, а также ссылка на предыдущий элемент. После заполнения массивов ответ восстанавливается обратным проходом по массиву *previous*.



```
package lab8.algo.task1;

import java.util.Arrays;

public class Solution {

    public int[] longestIncreasingSubsequence(int[] sequence) {
        if (sequence.length == 0) {
            return new int[0];
        }

        int[] length = new int[sequence.length];
        int[] previous = new int[sequence.length];
        Arrays.fill(length, 0);
        Arrays.fill(previous, -1);

        int bestEnd = 0;

        for (int current = 1; current < sequence.length; current++) {
            for (int candidate = 0; candidate < current; candidate++) {
                if (sequence[candidate] < sequence[current] &&
                    length[candidate] + 1 > length[current]) {
                    length[current] = length[candidate] + 1;
                    previous[current] = candidate;
                }

                if (length[current] > length[bestEnd]) {
                    bestEnd = current;
                }
            }

            int[] answer = new int[length[bestEnd]];
            int position = answer.length - 1;
            int current = bestEnd;
            while (current > -1) {
                answer[position] = sequence[current];
                position--;
                current = previous[current];
            }

            return answer;
        }

        public String format(int[] sequence) {
            int[] answer = longestIncreasingSubsequence(sequence);
            StringBuilder builder = new StringBuilder();
            for (int i = 0; i < answer.length; i++) {
                builder.append(' ');
                builder.append(answer[i]);
            }
            return builder.toString();
        }
    }
}
```



## 4.2. CodeRun. «Расстояние по Левенштейну»

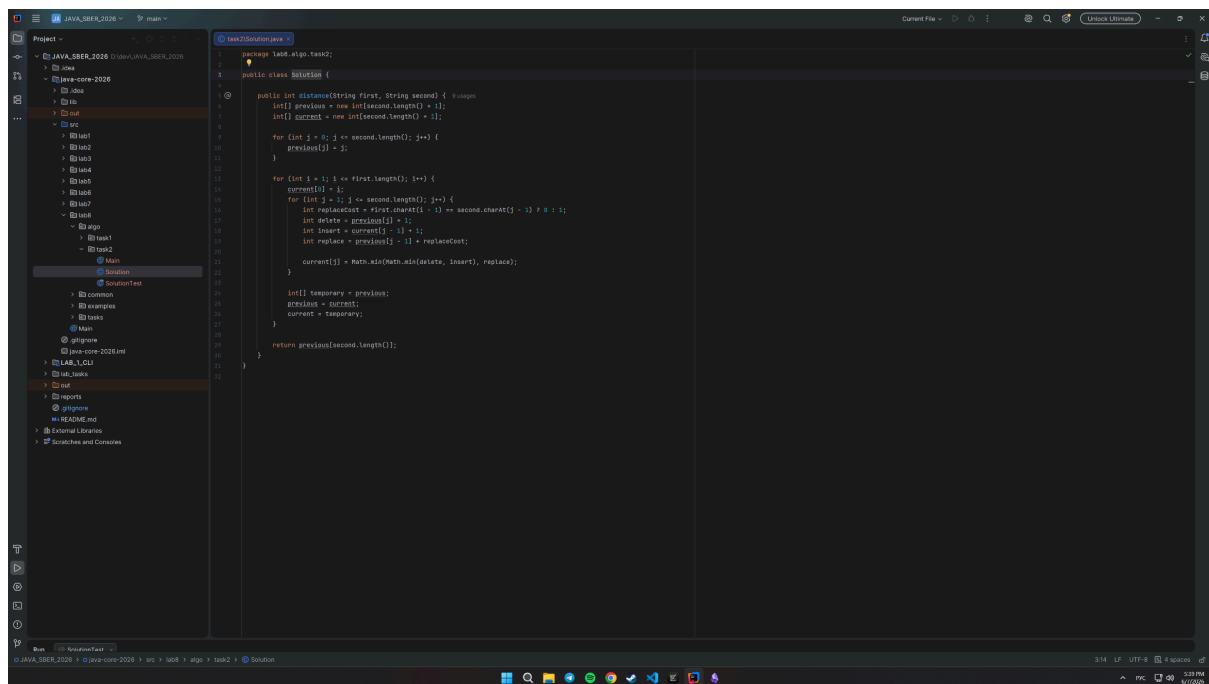
Файлы с решением:

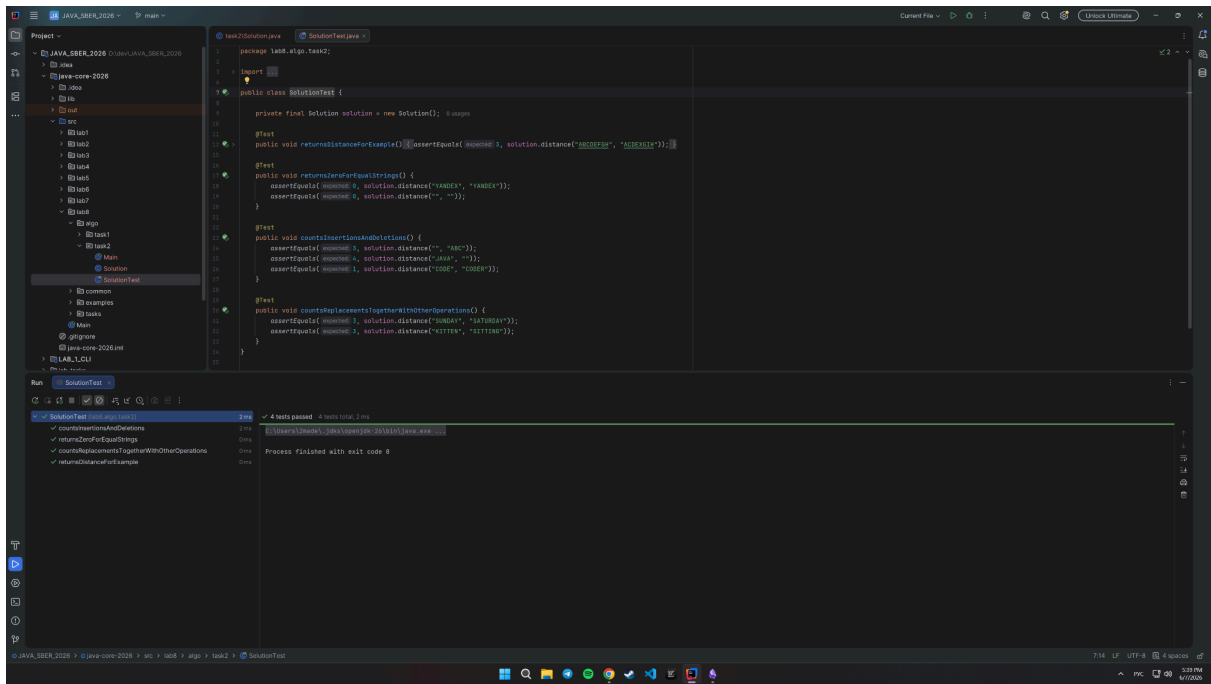
java-core-2026/src/lab8/algo/task2/Main.java

java-core-2026/src/lab8/algo/task2/Solution.java

java-core-2026/src/lab8/algo/task2/SolutionTest.java

По условию требуется найти минимальное количество операций вставки, удаления и замены символа для преобразования одной строки в другую. В решении используется динамическое программирование с двумя строками матрицы, поэтому память расходуется как  $O(m)$ , где  $m$  — длина второй строки.





### 4.3. Тестирование алгоритмических задач

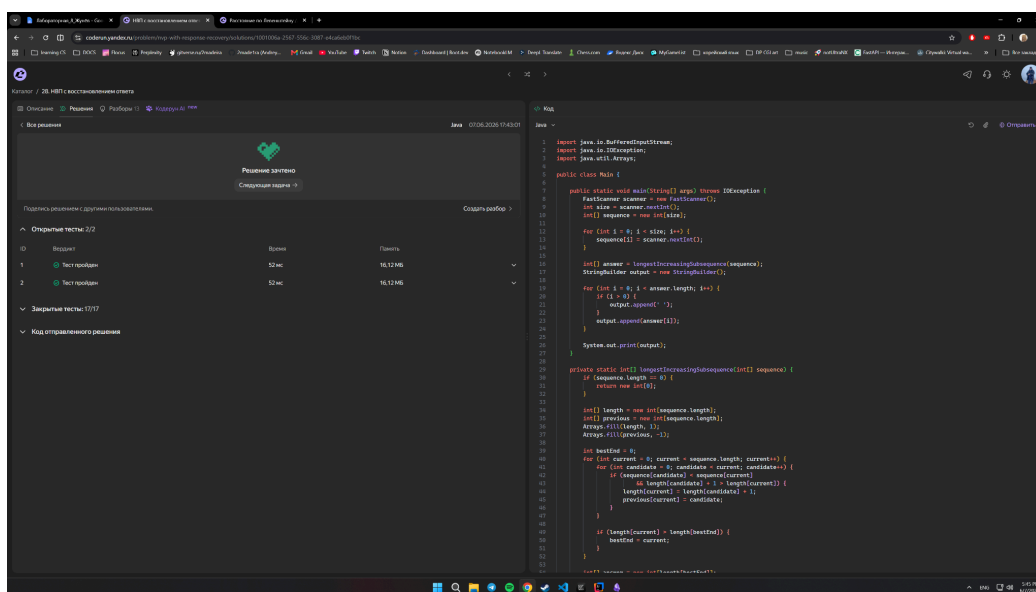
Для обеих CodeRun-задач подготовлены JUnit-тесты:

java-core-2026/src/lab8/algo/task1/SolutionTest.java

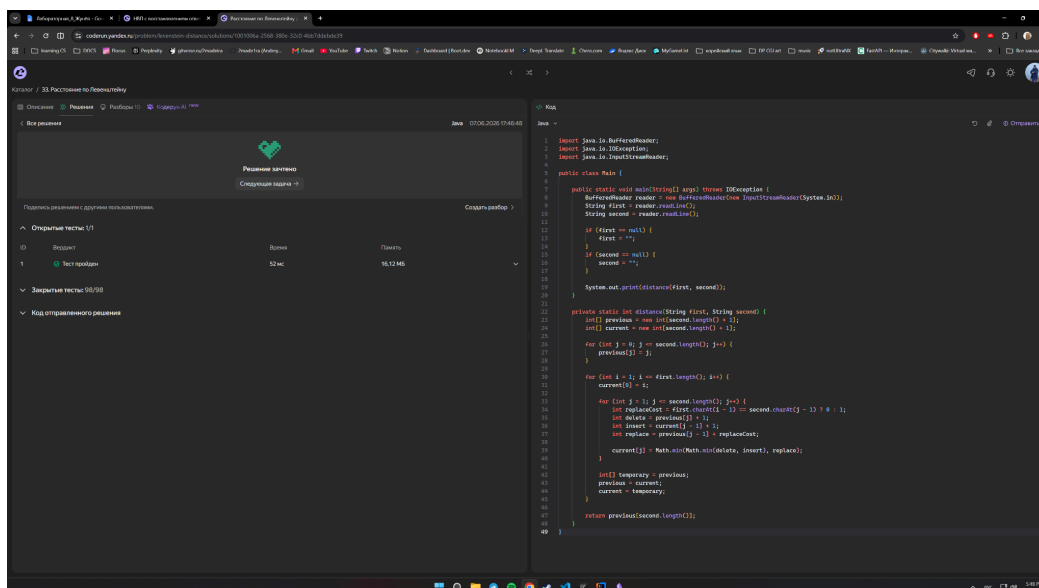
java-core-2026/src/lab8/algo/task2/SolutionTest.java

Решение принято на платформе CodeRun:

<https://coderun.yandex.ru/problem/nvp-with-response-recovery/description>



<https://coderun.yandex.ru/problem/levenstein-distance>



## Вывод

В ходе лабораторной работы были изучены и реализованы способы работы Java-программ с форматами XML, JSON, HTML и Excel. Для XML использован DOM/JAXP-парсер, позволяющий создавать, читать, изменять и сохранять дерево документа. Для JSON реализована простая файловая обвязка, которая создает массив книг, читает его, выполняет поиск, добавление и удаление записей. Для HTML реализованы загрузка страницы с повторными попытками, извлечение ссылок и новостей, а также сохранение результатов в файл. Для Excel реализовано создание и чтение XLSX-файла, включая обработку ошибок при отсутствии нужного листа.

Дополнительно были решены две алгоритмические задачи CodeRun: восстановление наибольшей возрастающей подпоследовательности и расчет расстояния Левенштейна. Для них подготовлены классы **Main**, **Solution** и JUnit-тесты. Компиляция всех файлов пакета **lab8** прошла успешно, тесты завершились результатом **OK (8 tests)**, демонстрационные программы вывели ожидаемые результаты.